ФГБОУ ВО «Пензенский государственный университет»
Факультет вычислительной техники
Кафедра «Вычислительная техника»

ОТЧЕТ

по дисциплине «Логика и ОА»
лабораторная работа №3 на тему:
«Оценка времени выполнения программ»

Выполнил студенты группы 25ВВВ3:
Коновалов А. С
Шорохов В. В
Приняли:
к.т.н., доцент Юрова О. В.
к.т.н., доцент Деев М. В.

Пенза 2026

Тема: «Оценка времени выполнения программ»

Цель работы: изучить методы оценки времени выполнения программ на языке C с использованием библиотеки <time.h>. Освоить функции clock(), time(), difftime() и константу CLOCKS_PER_SEC. Экспериментально определить время выполнения умножения матриц различных размеров и алгоритмов сортировки (Шелла, быстрой сортировки и стандартной qsort) на разных наборах данных. Оценить вычислительную сложность алгоритмов, построить графики зависимости времени от размера задачи и типа входных данных, сравнить экспериментальные результаты с теоретическими оценками.

Лабораторное задание:

Задание 1:

1. Вычислить порядок сложности программы (O-символику).

2. Оценить время выполнения программы и кода, выполняющего перемножение матриц, используя функции библиотеки time.h для матриц размерами от 100, 200, 400, 1000, 2000, 4000, 10000.

3. Построить график зависимости времени выполнения программы от размера матриц и сравнить полученный результат с теоретической оценкой.

Задание 2:

1. Оценить время работы каждого из реализованных алгоритмов на случайном наборе значений массива.

2. Оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой возрастающую последовательность чисел.

3. Оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой убывающую последовательность чисел.

4. Оценить время работы каждого из реализованных алгоритмов на массиве, одна половина которого представляет собой возрастающую последовательность чисел, а вторая, – убывающую.

5. Оценить время работы стандартной функции qsort, реализующей алгоритм быстрой сортировки на выше указанных наборах данных.

Пояснительный текст:

Задание 1. Оценка времени перемножения матриц

Программа предназначена для измерения времени выполнения операции умножения двух квадратных матриц различных размеров. Для оценки времени используется библиотека <time.h> и функция clock(), возвращающая количество тактов процессора с момента запуска программы.

Алгоритм работы

Функция multiply_matrices(int N) — основная вычислительная функция. Она принимает размер матрицы N и возвращает время перемножения в секундах (тип double), либо -1.0 в случае ошибки выделения памяти.

Выделение **памяти.** Под три матрицы a, b и c размера N×N динамически выделяется память через malloc. Матрицы хранятся в виде одномерных массивов размером N·N элементов, а доступ к элементу с индексами (i, j) осуществляется по формуле a[i * N + j]. Такой способ позволяет одним вызовом malloc выделить память под всю матрицу, что эффективнее, чем N отдельных вызовов для каждой строки. Приведение к типу (size_t) перед умножением предотвращает переполнение при больших N (например, для N = 10000 произведение N·N превышает диапазон типа int).

Проверка выделения памяти. Если хотя бы один из указателей равен NULL, программа выводит сообщение «ОШИБКА ПАМЯТИ при N = …», освобождает уже выделенную память и возвращает -1.0.

Инициализация **матриц.** В цикле по всем N·N элементам массивы a и b заполняются случайными числами в диапазоне от 1 до 100 (rand() % 100 + 1). Генератор инициализируется текущим временем через srand(time(NULL)).

Замер времени. Перед вычислениями сохраняется начальное значение счётчика тактов: start = clock().

Умножение матриц. Реализовано классическим тройным вложенным циклом:

- внешний цикл по i (строки матрицы a),
- средний цикл по j (столбцы матрицы b),
- внутренний цикл по r — накопление суммы произведений a[i][r] * b[r][j] в переменной elem_c.

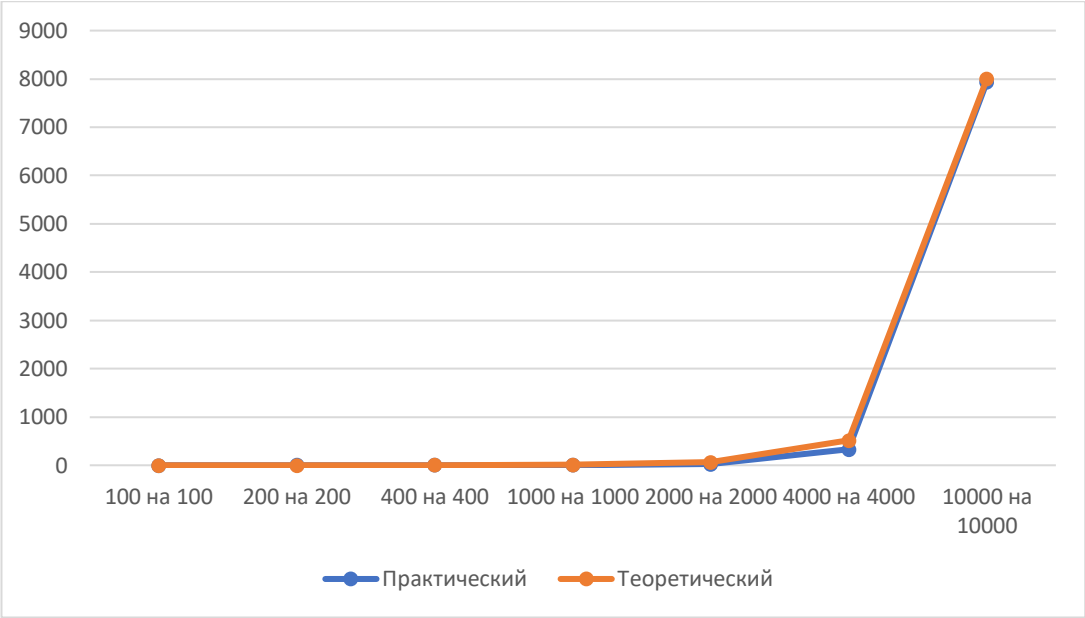
После завершения внутреннего цикла результат записывается в c[i][j]. Вычислительная сложность алгоритма — $O(N^3)$, так как выполняется N^3 операций умножения и сложения.

Конец замера. Сохраняется end = clock(), память освобождается через free, и функция возвращает (double)(end - start) / CLOCKS_PER_SEC — время в секундах.

Функция main. Задаёт массив размеров {100, 200, 400, 1000, 2000, 4000, 10000} и последовательно вызывает multiply_matrices для каждого размера, сохраняя результаты в массиве results[7]. Перед запуском выводится сообщение «Идет расчет для N = …», так как для больших N вычисления могут занимать минуты. После всех замеров результаты выводятся в виде таблицы с рамкой. Если для какого-то размера вернулось отрицательное значение (ошибка памяти), выводится строка «ОШИБКА ПАМЯТИ».

Русификация вывода. В начале main вызываются setlocale(LC_ALL, "Russian"), SetConsoleCP(1251) и SetConsoleOutputCP(1251) — это обеспечивает корректное отображение русских символов в консоли Visual Studio.

График времени исполнения программы:



Результат работы программы:

```
=== ЗАДАНИЕ 1: Измерение времени перемножения матриц ==

Идет расчет для N = 100 (пожалуйста, подождите)...
Идет расчет для N = 200 (пожалуйста, подождите)...
Идет расчет для N = 400 (пожалуйста, подождите)...
Идет расчет для N = 1000 (пожалуйста, подождите)...
Идет расчет для N = 2000 (пожалуйста, подождите)...
Идет расчет для N = 4000 (пожалуйста, подождите)...
Идет расчет для N = 10000 (пожалуйста, подождите)...


ИТОГОВАЯ ТАБЛИЦА РЕЗУЛЬТАТОВ:
+--------------------+--------------------+
|  Размер матрицы    |  Время счета (сек) |
+--------------------+--------------------+
|    100  x100       |           0.00     |
|    200  x200       |           0.01     |
|    400  x400       |           0.11     |
|    1000 x1000      |           2.52     |
|    2000 x2000      |          23.90     |
|    4000 x4000      |         329.29     |
|    10000x10000     |        7937.78     |
+--------------------+--------------------+
```

Задание 2:

Программа сравнивает время работы трёх алгоритмов сортировки — сортировки Шелла (shell), быстрой сортировки (qs_wrap) и стандартной

библиотечной функции qsort — на четырёх типах входных данных и трёх размерах массива.

Алгоритм работы

Сортировка Шелла (shell). Реализует классический алгоритм с фиксированной последовательностью зазоров {9, 5, 3, 2, 1}. Для каждого зазора gap выполняется сортировка вставками с этим шагом: элемент items[i] запоминается в x, затем во внутреннем цикле все элементы, большие x, сдвигаются на gap позиций вправо, после чего x вставляется на нужное место. Алгоритм модифицирует массив «на месте», не требуя дополнительной памяти. Сложность в среднем — O(N^1.3).

Быстрая сортировка (qs). Классическая рекурсивная реализация схемы Хоара. Опорный элемент выбирается как средний элемент items[(left + right) / 2]. Два указателя i и j движутся навстречу друг другу, обменивая элементы, стоящие не на своих местах. После разделения рекурсивно сортируются левая и правая части. Обёртка qs_wrap вызывает qs(items, 0, n-1) — так удобно передавать функцию в measure. Средняя сложность — O(N log N), худшая — $O(N^2)$ (например, на уже отсортированном массиве при неудачном выборе опорного элемента).

Стандартная qsort. Использует библиотечную функцию из <stdlib.h>, которой передаётся функция сравнения compare_ints. Эта функция возвращает (ia > ib) - (ia < ib), что соответствует требованиям qsort и корректно обрабатывает все случаи. Обёртка qsort_wrap делает сигнатуру совместимой с указателем на функцию void (*)(int*, int).

Функция fill_art заполняет массив заданного размера n значениями одного из четырёх типов:

- type == 0 — случайные числа от 0 до 9999 (rand() % 10000);
- type == 1 — возрастающая последовательность 0, 1, 2, ...;
- type == 2 — убывающая последовательность n, n-1, n-2, ...;
- type == 3 — «пик»: первая половина возрастает (0, 1, 2, ...), вторая убывает (n - i).

Функция measure. Измеряет время работы переданной функции сортировки:

- выделяет память под копию массива copy,
- копирует исходный массив через memcpy (чтобы каждый алгоритм сортировал одинаковые данные),
- засекает start = clock(),
- вызывает сортировку,
- засекает end = clock(),
- освобождает память и возвращает время в секундах.

Функция main. Для каждого размера n ∈ {100, 1000, 10000}:

- создаёт четыре исходных массива (orig_rand, orig_asc, orig_desc, orig_peak) с соответствующими типами данных,
- последовательно для каждого алгоритма вызывает measure на каждом из четырёх массивов,
- выводит результаты в виде таблицы.

Таблица имеет столбцы «Случайный», «Возраст.», «Убывающ.», «Пик» и строки «Shell», «qs», «qsort». Для каждого размера печатается отдельная таблица с рамкой.

Русификация вывода. В начале main вызываются setlocale(LC_ALL, "Russian"), SetConsoleCP(1251) и SetConsoleOutputCP(1251) — это обеспечивает корректное отображение русских символов в консоли Visual Studio.

Таблица 1. Время сортировки (N = 100)

Алгоритм	Случайный (сек)	Возрастающий (сек)	Убывающий (сек)	Пик (сек)
Shell	0.0000	0.0000	0.0000	0.0000
qs	0.0000	0.0000	0.0000	0.0000

qsort	0.0000	0.0000	0.0000	0.0000

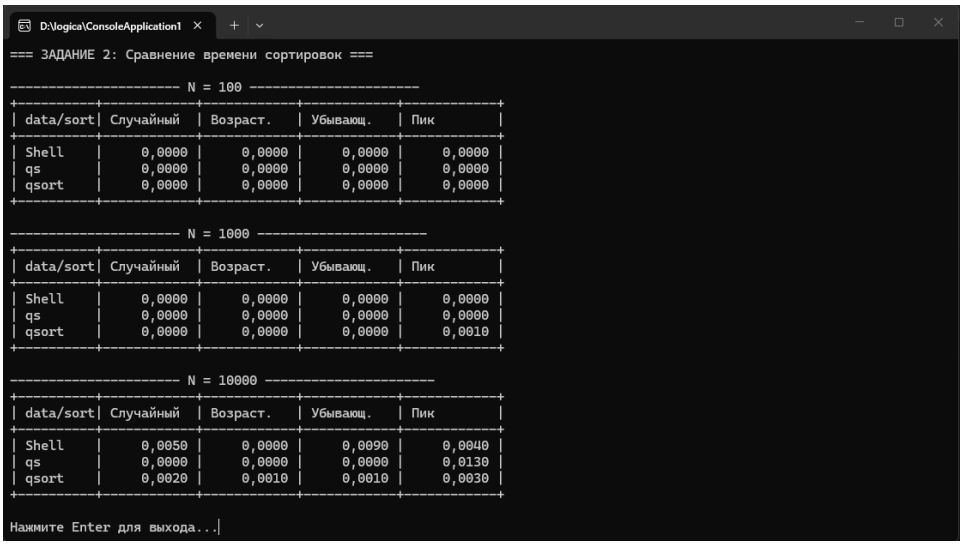
Таблица 2. Время сортировки (N = 1000)

Алгоритм	Случайный (сек)	Возрастающий (сек)	Убывающий (сек)	Пик (сек)
Shell	0.0000	0.0000	0.0010	0.0000
qs	0.0000	0.0000	0.0000	0.0000
qsort	0.0000	0.0000	0.0000	0.0010

Таблица 3. Время сортировки (N = 10000)

Алгоритм	Случайный (сек)	Возрастающий (сек)	Убывающий (сек)	Пик (сек)
Shell	0.0050	0.0000	0.0090	0.0040
qs	0.0000	0.0000	0.0000	0.0130
qsort	0.0020	0.0010	0.0010	0.0030

Результат работы программы:



Вывод:

В ходе лабораторной работы были освоены практические приёмы измерения времени выполнения программ средствами библиотеки <time.h>.

Для задачи умножения матриц экспериментально подтверждена кубическая сложность: $O(n^3)$ при увеличении размера матрицы время выполнения возрастает пропорционально кубу размера. Для алгоритмов сортировки установлено, что время работы существенно зависит как от самого алгоритма, так и от исходного порядка элементов.